SLARC Pi↔ESP32 Protokoll v0.1

Oct 17, 2026 · @Marco

Transport und Rahmen

UART über den 40-Pin-Header (Pi GPIO14/15 ↔ ESP32), 921600 Bd 8N1, eine JSON-Nachricht pro Zeile (NDJSON), abgeschlossen mit `\n`, max. 256 Byte. Baudrate auf dem Waveshare-Board erst verifizieren; Fallback 460800.

Jede Zeile endet mit einer CRC-8 nach NMEA-Art: `{...}*A3\n` (CRC über die Bytes zwischen `{` und `}` inklusive). Fehlerhafte Zeilen werden verworfen und mit `nak` / `E_CRC` beantwortet, sofern eine `id` lesbar ist.

Feld	Typ	Bedeutung
`t`	string	Nachrichtentyp, kurz (`mv` , `leg` , `tel` …)
`id`	uint16	Laufende Nummer vom Pi, Wrap bei 65535; Antworten tragen dieselbe `id`
`p`	int	Optional: Preempt, 1 = laufende Aktion ersetzen

Einheiten: mm, mm/s, Grad, Grad/s, N, ms – alles als Integer, Winkel in der Telemetrie als 0,1°. Körperkoordinaten nach REP-103: x vorne, y links, z oben, Ursprung Körpermitte auf Höhe der Coxa-Achsen. Keine Weltkoordinaten (Architekturregel).

Beine: `FR MR HR FL ML HL` bzw. Index 0–5 in dieser Reihenfolge; Bitmasken nutzen dieselbe Reihenfolge (Bit 0 = FR).

Befehlsklassen

Drei Klassen, weil "ein Arbeitspaket nach dem anderen" für Laufen nicht passt: Laufen ist ein Sollwertstrom, Greifen ist eine Aktion mit Ende.

Klasse	Verhalten	Antwort	Timeout
S – Stream	Letzter Wert gilt, keine Queue	nur `nak` bei Fehler	Kein neuer Wert in 300 ms → Sollwert auf 0 rampen
A – Aktion	Genau eine aktiv; neue während aktiv → `nak` `E_BUSY` , außer `p:1`	`ack` sofort, `done` am Ende	Aktionsspezifisch (`to` in ms, Default 10 s)

Klasse	Verhalten	Antwort	Timeout
C – Konfig	Wird sofort übernommen, wirkt ab nächstem Zyklus	`ack` / `nak`	–

Regel: Während eine A-Aktion mit Beinen läuft, werden S-Befehle für Laufen mit `E_MODE` abgelehnt. Body/Attitude-Streams bleiben erlaubt, damit der Pi beim Greifen den Körper nachführen kann.

Befehle Pi → ESP32

Deine Punkte 1–6 sind enthalten, ergänzt um alles, was für Fernsteuerung, Greifen und sicheres Abschalten zusätzlich nötig ist (markiert mit *neu*).

`t`	Kl.	Parameter	Zweck
`mv`	S	`vx` `vy` mm/s, `wz` °/s	(1) Laufen. Kontinuierlich statt 6 Richtungen; die Teleop-Tasten setzen feste Werte
`body`	S	`z` mm, `x` `y` mm (Körperverschiebung)	(2) Körperhöhe als relative Beinstreckung + *neu* Schwerpunktverschiebung
`att`	S	`pitch` `roll` `yaw` °	(3) Pitch + *neu* Roll (nötig für das Tripod-Entlasten beim Klettern) und Yaw im Stand
`leg`	A	`leg`, `x` `y` `z` mm, `v` mm/s, `mode`: `pos` / `guard`, `fmax` N, `to` ms	(4) Fußspitze anfahren. `guard` stoppt bei µ-Schalter oder Kraftschwelle und meldet Kontaktposition
`bal`	C	`on` 0/1, optional `kp`	(5) IMU-Balance an/aus
`gait`	C	`type`: `tripod` / `ripple` / `wave` / `probe` / `climb`, `sh` Schritthöhe mm, `tc` Zykluszeit ms	(6) Gangart. `probe` = bisheriger Mode 7 (Tast-Gang)
`pose`	A	`name`: `relax` / `park` / `sit` / `stand` / `manip`	*neu* Feste Haltungen. `manip` verlagert den Schwerpunkt auf die hinteren vier Beine und gibt FL/FR frei

`t`	Kl.	Parameter	Zweck
`grip`	A	`legs` (z.B. `["FL","FR"]`), `w` Zielbreite mm, `f` Haltekraft N, `fmax` N, `v` mm/s, `h` Greifhöhe mm	*neu* Kraftbegrenztes Greifen, siehe letzter Abschnitt
`rel`	A	`legs`, `w` Öffnungsbreite mm	*neu* Loslassen
`stop`	A	–	*neu* Weich anhalten: Geschwindigkeiten auf 0 rampen, Schwungbeine absetzen
`estop`	–	–	*neu* Sofort einfrieren mit Haltemoment. Nie Drehmoment aus im Stand
`abort`	–	`id` der Aktion	*neu* Laufende Aktion abbrechen, aktuelle Position halten
`hb`	–	`n` Zähler	*neu* Heartbeat, alle 100 ms
`arm`	C	`on` 0/1	*neu* Freigabe für Bewegung; nach Boot und nach Heartbeat-Verlust immer 0
`cfg`	C	`tel` Hz, `tlim` Moment-Limits je Gelenk %, `vmax`, `zmin` `zmax`	*neu* Grenzen und Telemetrierate
`hello`	–	`v` Protokollversion	*neu* Versionsabgleich beim Verbinden; Mismatch → ESP bleibt unarmed
`srv`	A	`j` Gelenk 0–17, `q` 0,1°, `v`	*neu* Rohes Gelenk anfahren, nur im Modus `relax` / `debug` – für Kalibrierung
`off`	A	–	*neu* Shutdown-Einleitung, siehe Sicherheit

Beispiele:

```
{"t":"mv","id":412,"vx":80,"vy":0,"wz":0}*5C
{"t":"leg","id":413,"leg":"FL","x":260,"y":120,"z":-40,"v":30,"mode":"guard",
"fmax":8}*1E
{"t":"grip","id":414,"legs":
["FL","FR"],"w":66,"f":4,"fmax":10,"v":10,"h":-60}*9B
```

Offener Punkt: Ob `leg` mit `x y z` im Körperframe oder im Coxa-Frame des Beins arbeitet. Vorschlag Körperframe, weil Greifen mit zwei Beinen dann symmetrisch formulierbar ist.

Nachrichten ESP32 → Pi

`t`	Wann	Inhalt
`ack` / `nak`	Innerhalb 5 ms nach Empfang	`id`, bei `nak` zusätzlich `e` Fehlercode
`done`	Ende einer A-Aktion	`id`, `res`: `ok` / `fail` / `abort` / `timeout`, `e`, Ergebnisdaten (z.B. Kontaktposition bei `guard`, gemessene Breite bei `grip`)
`tel`	50 Hz (einstellbar)	`ts` ms, `st` Zustand, `ph` Gangphase, `q[18]` 0,1°, `c` Kontakt-Bitmaske, `rp` Roll/Pitch 0,1°, `gz` Gierrate, `act` aktive Aktions-`id`
`tel2`	2 Hz	`ld[18]` Last %, `tmax` °C, `vb` mV, `err` Servo-Fehlerbits
`evt`	Ereignisgesteuert	`k`: `contact`, `tilt`, `vlow`, `ovtemp`, `ovload`, `grip_contact`, `grip_slip`, `hb_lost`, `servo_lost`
`hb`	Alle 100 ms	`n` Echo des letzten Pi-Zählers, `st` Zustand – Pi erkennt daran ESP-Ausfall

Zustände `st`: `boot`, `relax`, `park`, `stand`, `walk`, `manip`, `safe`, `fault`.

Code	Bedeutung
`E_CRC`	Prüfsumme falsch
`E_PARSE`	JSON ungültig / Feld fehlt
`E_RANGE`	Wert außerhalb `cfg`-Grenzen oder Arbeitsraum
`E_BUSY`	Aktion läuft, kein Preempt
`E_MODE`	Im aktuellen Zustand nicht erlaubt (z.B. `mv` in `manip`)
`E_ARM`	Nicht armed
`E_STAB`	Würde Stützpolygon verlassen

Code	Bedeutung
`E_SERVO`	Servo antwortet nicht / Fehlerbit
`E_LOAD`	Momentgrenze erreicht

Bandbreite: `tel` ca. 200 Byte × 50 Hz = 10 kB/s, bei 921600 Bd etwa 11 % Auslastung. Falls es eng wird, `q[]` später binär (Base64) packen.

Sicherheit: Heartbeat, Safe-Stop, Shutdown

Der ESP32 entscheidet Sicherheit lokal und wartet dabei nie auf den Pi.

Ausfallzeit Pi-Heartbeat	Reaktion ESP32
> 300 ms	`evt hb_lost`, Streams auf 0 rampen, laufende Schwungbeine absetzen, `arm` = 0
> 2 s	Zustand `safe`: Körper auf `zmin` absenken, alle Füße am Boden, Haltemoment an
Während `grip`	Griff halten, nicht öffnen, Beine einfrieren – Dose fällt nicht

Nach Rückkehr des Heartbeats läuft nichts automatisch weiter: Der Pi muss `hello` und `arm on` neu senden. Alte `mv`-Werte sind verworfen.

Lokale Schutzfunktionen ohne Pi: Momentgrenze je Gelenk (Servo-Register + Software), Übertemperatur, Unterspannung, Kippwinkel aus IMU, Arbeitsraumgrenzen der IK.

Shutdown-Ablauf (passend zur beschlossenen Stromsequenz):

1. Pi sendet `pose park`, ESP meldet `done ok`.
2. Pi sendet `off`, ESP quittiert und wartet auf Pi-Abschaltung.
3. Pi fährt herunter; ESP erkennt Heartbeat-Ende, hält Park-Haltung, schaltet dann Servo-Moment aus.
4. Schiebeschalter trennt alles.

Beim Boot startet der ESP in `relax` oder `park`, `arm` = 0, und bewegt nichts ohne `hello` + `arm on` + `pose stand`.

Greifen ohne Zerquetschen

Drei Schichten: weiche Pads senken die nötige Kraft, das Servo-Momentlimit begrenzt sie hart, und der ESP regelt die Kraft nach Kontakt.

Nötige Klemmkraft bei zwei gegenüberliegenden Kontakten, volle 0,33-l-Dose (ca. 0,36 kg, Gewicht 3,5 N):

```
F_N \geq S \cdot \frac{m g}{2 \mu}
```

Pad-Material	μ auf Alu (grob)	F_N mit S = 2
Silikon/TPU weich	0,8	ca. 4,4 N
PLA blank	0,25	ca. 14 N

Zum Vergleich: ST3215 mit 2,94 Nm am Hebel 0,25 m schafft nur ca. 12 N an der Fußspitze. Ohne Reibbeläge reicht die Kraft also kaum, mit Belägen gibt es Reserve. Die Quetschgrenze der leeren, offenen Dose ist unbekannt – mit Küchenwaage und Fingerdruck messen, bevor `fmax` festgelegt wird. Volle, geschlossene Dosen sind durch den Innendruck deutlich steifer.

Maßnahmen:

1. Mechanik: Entschieden: seitliche TPU-Pads mit Riffelung an den Beinen. Rillen quer zur Schwerkraft, damit sie gegen vertikales Rutschen greifen. Kontakt am Pad liegt nicht am μ-Schalter der Fußspitze, daher Kontakterkennung beim Greifen nur über Last bzw. Positionsabweichung. Nahe dem oberen oder unteren Rand greifen, dort ist die Dose steif; nicht in der Mantelmitte.
2. Hartes Limit: Vor dem Schließen das Register `Torque_Limit` der beteiligten ST3215 auf einen kalibrierten Wert setzen. Das greift auch, wenn die Software falsch rechnet.
3. Kontakt finden: Langsam schließen (ca. 10 mm/s) im `guard`-Modus. Kontakt = Lastsprung oder Abweichung Soll- zu Ist-Position.
4. Kraft halten (einfache Variante zuerst): Sollposition einige mm hinter den Kontaktpunkt legen; der Servo läuft ins Momentlimit und drückt mit begrenzter Kraft. Später verfeinern mit Kraftschätzung über die Jacobi-Matrix: $F = J^{-T} \cdot (\tau_mess - \tau_schwer)$.
5. Kalibrieren: Tip-Kraft gegen Momentlimit in % mit Küchenwaage in der Greifpose messen, als Tabelle im ESP hinterlegen. Die Lastanzeige der ST3215 ist grob und verrauscht.

6. Schlupf: Mit niedriger Kraft `f` beginnen, 1 cm anheben, Schlupf prüfen (Kamera oder Positionsänderung der Greifbeine), dann schrittweise bis `fmax` erhöhen. Ob voll oder leer, weiß der Pi vorher nicht – so findet der Griff die kleinste ausreichende Kraft.

Voraussetzung: Vor `grip` immer `pose manip`, damit der Roboter stabil auf vier Beinen steht.